Relatório Técnico: GenResearch-Servidor - Sistemas Operacionais - Atividade 2

Autora: Mariana Silva Costa
Data: 20/10/2026
Repositório: https://github.com/Kwttni/GenResearch-Servidor
Vídeo de Apresentação: GenResearch-Servidor - Mariana Costa

1. Introdução e Problema

Atualmente aplicações de IA generativa e Recuperação Aumentada por Geração (RAG) exigem muito do Sistema Operacional. Elas consomem muita memória física para carregar o contexto, usam memória virtual para lidar com arquivos gigantes, realizam operações pesadas de I/O na leitura de índices vetoriais e dependem muito de concorrência e filas.

Este relatório analisa o funcionamento do serviço simplificado, o GenResearch-Servidor. A intenção do projeto é responder diretamente à pergunta central da atividade:

"Como memória principal, memória virtual, armazenamento, caches, filas compartilhadas e mecanismos de sincronização afetam o desempenho, a estabilidade e a correção de uma aplicação de IA generativa ou de RAG?"

2. Registo de Configuração do Sistema e Ambiente

2.1. Hardware

Tipo de ambiente: Nativo
Sistema Operacional: Fedora Linux 44 (Workstation Edition)
Kernel: Linux 7.2.9-200.fc44.x86_64
CPU: AMD Ryzen 5 5600GT with Radeon Graphics (6 núcleos, 12 threads)
RAM: 15 GB Total (8,3 GB disponíveis para novas alocações)
Armazenamento: SSD NVMe 512GB (nvme0n1) com 337 GB livres
Swap: 8 GB em /dev/zram0 (apenas 14 MB em uso)
CPU: Gráficos integrados Radeon (compartilham RAM do sistema)
Sistema de arquivos: Btrfs (diretório raiz e de utilizador partilham o mesmo subvolume no armazenamento)

2.2. Ferramentas

Linguagem: Python 3.14.8
Bibliotecas e versões:
numpy 2.4.6
pandas 3.0.6
matplotlib 3.11.2
ollama 0.6.3
threading e multiprocessing

mmap
resource
time
sqlite 3
Commit do repositório: 2bdd08078f9b3d4b10e59de09d6b1bb18fd5ef4e
Data: 02/10/2026 às 17:57

2.3. Comandos Utilizados

Para executar o Makefile e realizar os testes:

```
make preparar        # Inicia a ingestão dos dados e a vetorização
make monitorar       # realiza os experimentos e captura as métricas
make evidencias-so   # coleta o rastreamento de threads e syscalls
make graficos        # processa o arquivo brutos.csv e gera os gráficos
```

Para monitorar os recursos e I/O:

```
vmstat -t 1      # captura o estado da memória virtual, swap e page fault em 1
                   segundo
iostat -xz 1     # monitora a vazão e latência de I/O no SSD
free -h          # registra o estado da memória física
```

Para rastrear processos e syscalls:

```
# lista e monitora o consumo de CPU/RAM das threads
ps -L -p <PID>
top -H -b -n 1 -p <PID>

# contabilza e sumeriza as syscalls pelo processo e por suas threads filhas
strace -f -c

# aplica limites na memória para testar o comportamento no sistema no C5
systemd-run --user --scope -p MemoryMax=1G…
```

3. Fundamentação e Análise de Gestão de Memória

3.1. MemFree, MemAvailable, Cache e Swap

- **MemFree:** Parte da memória RAM física que não está sendo utilizada no momento.

- **MemAvailable:** Estimativa da quantidade de memória disponível para executar novos programas sem recorrer à swap. Considera a memória livre e parte da memória ocupada por caches e outras páginas que podem ser liberadas sem comprometer o funcionamento do sistema.

- **Cache de páginas:** Parte da RAM utilizada para armazenar temporariamente dados de arquivos acessados, permitindo leituras mais rápidas ao evitar novos acessos ao armazenamento.

- **Swap:** Espaço de armazenamento utilizado como memória virtual. Permite transferir páginas da memória RAM para o disco quando necessário, liberando espaço na memória física. Como o armazenamento é mais lento que a RAM, o uso frequente de swap pode prejudicar o desempenho.

3.2. Papel da cache de páginas na memória

O cache de páginas não representa um desperdício de RAM, pois o kernel utiliza parte da memória disponível para armazenar dados de arquivos acessados. Quando os processos precisam de mais memória, essas páginas podem ser liberadas para dar espaço a outras necessidades do sistema.

O uso do cache permite acessar dados com mais rapidez, reduzindo a necessidade de novas leituras do armazenamento. Dessa forma, o kernel aproveita melhor a memória RAM disponível, embora nem todas as páginas em cache possam ser liberadas imediatamente.

3.3. Relevância da paginação e leitura de arquivos em datasets, índices, embeddings e checkpoints

Aplicações de IA generativa movimentam grandes volumes de dados, o que pode aumentar a demanda por memória RAM e operações de entrada e saída. O cache de páginas permite que dados acessados repetidamente sejam lidos mais rapidamente, reduzindo a latência de acesso ao armazenamento e podendo melhorar o desempenho das aplicações.

Em datasets: Armazenam dados utilizados no treinamento, na análise ou na avaliação dos modelos. Quando os mesmos arquivos são lidos repetidamente, o cache pode mantê-los na RAM, reduzindo a necessidade de novas leituras no disco.

Em índices: Estruturas utilizadas para organizar e recuperar informações, como os índices vetoriais empregados em sistemas RAG. O cache pode acelerar a leitura dos arquivos de índice durante consultas repetidas, especialmente quando eles são grandes.

Em embeddings: Representações numéricas de textos ou outros dados, utilizadas para comparar similaridades e recuperar informações relevantes. Quando armazenados em arquivos e acessados repetidamente, os embeddings podem se beneficiar do cache, reduzindo o tempo de leitura dos vetores.

Em checkpoints: Arquivos que salvam o estado de um modelo durante o treinamento, incluindo seus pesos e, dependendo da implementação, outras

informações necessárias para retomar a execução. O cache pode beneficiar a leitura desses arquivos e, em determinadas condições, ajudar a melhorar o desempenho das operações de gravação.

4. Arquitetura do Sistema

A arquitetura do GenResearch-Servidor foi concebida com um fluxo clássico de produtor-consumidor:

1. **Ingestão e Vetorização:** O sistema lê os documentos, divide em pedaços (chunks) e cria os embeddings (usando um hash determinístico para gerar o teste de estresse do SO).

2. **Armazenamento:** Salva os textos no SQLite (genresearch.db) e o índice de vetores no NumPy (vetores.npy).

3. **Fila Compartilhada (Buffer):** As perguntas entram numa fila controlada por semáforos, o que ajuda a não sobrecarregar.

4. **Recuperação e Resposta:** As threads consumidoras pegam as perguntas, calculam a similaridade dos vetores, pegam o texto do banco e geram a resposta.

5. Dados, Corpus, Embeddings, Índice e Cache

5.1. Dados

O dataset usado para o projeto foi extraído do site Kaggle e é nomeado como “Dataset Sephora Products and Skincare Reviews”. Nele contém reviews de produtos da loja Sephora. Apenas o CSV “reviews_0-250” foi usado para o projeto .

Disponível em:
https://www.kaggle.com/datasets/nadyinky/sephora-products-and-skincare-reviews

5.2. Corpus, Embedding, Índice e Cache

O arquivo “ingestao.txt” coletou dados da etapa de Ingestão do processo, nela foram obtidos os seguintes resultados:

Documentos processados pelo índice vetorial: 5000
Trechos divididos: 6062
Banco: genresearch.db
Índice: vetores.npy
Metadados: meta.json

Os vetores têm dimensão de 1024 e ficam salvos no disco. Para não gerar retrabalho, foi colocado uma Cache na memória RAM usando a política LRU. Se a capacidade de itens for atingida, o sistema descarta a busca mais antiga para dar espaço à nova.

6. Processos, Threads, Filas e Recursos Partilhados

A aplicação foi configurada para criar concorrência através do uso da biblioteca *threading*. Foram criadas threads separadas para envio de requisições (produtores) e para quem as processa (consumidores).

O recurso principal partilhado é a fila (collections.deque), cuja capacidade foi limitada para observar o bloqueio produtivo (backpressure) em cenários onde os consumidores estão saturados com operações de I/O. O cache LRU também é um recurso partilhado por todas as trabalhadoras.

7. Estratégia de Sincronização

Foi utilizado mutex para evitar problemas de segmentação ou corrupção de estado, tendo em vista que várias threads leem e escrevem na Cache ao mesmo tempo.

Foram colocados mutex (threading.Lock()) nas funções de adicionar e buscar itens na Cache, a fim de garantir que duas requisições não sobrescrevam a mesma chave no mesmo exato tempo.

Na fila, além do Mutex para o *append* e o *popleft*, foram usados dois semáforos (vazios e cheios). Dessa forma, as threads são colocadas para dormir quando a fila está cheia (ou vazia) e acordam só quando necessário, economizando CPU.

8. Metodologia

Cada configuração foi repetida 3 vezes para eliminar anomalias. As métricas base de tempo, *RSS* e faltas de página foram recolhidas com auxílio das ferramentas time, módulo resource do Python, e dos comandos de monitorização em background *vmstat* e *iostat*.

9. Resultados e Análise: Memória, Armazenamento, Paginação e I/O

9.1. Experimentos C1 e C2: Leitura Convencional vs Mapeamento de Memória - mmap

O experimento avaliou as diferenças de desempenho e gestão de memória virtual entre leituras de arquivo padrão e chamadas mmap, conforme exigido para a análise de armazenamento e I/O.

Para leituras a frio de arquivos grandes (512MB) de forma sequencial, o método convencional (open()/read()) registrou tempo de leitura de 0.227s.

A leitura via mmap na mesma configuração apresentou um tempo de leitura ligeiramente maior de 0.2445s, porém transferiu o custo para o sistema de paginação, gerando 1 falta de página grave (majflt) e 593 faltas menores (minflt).

Em consultas a quente (dados em cache), o mmap demonstrou melhor desempenho significativo para índices estruturados, reduzindo o tempo de leitura sequencial do indice_npy de 0.002573s (convencional) para 0.0005843s (mmap).

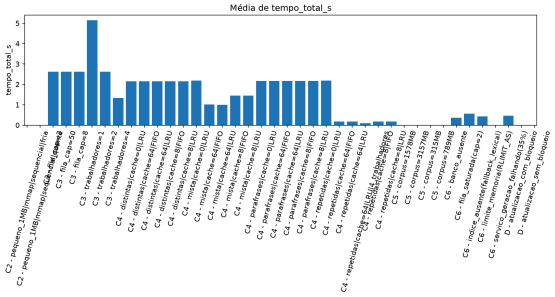


Gráfico de tempo de leitura.

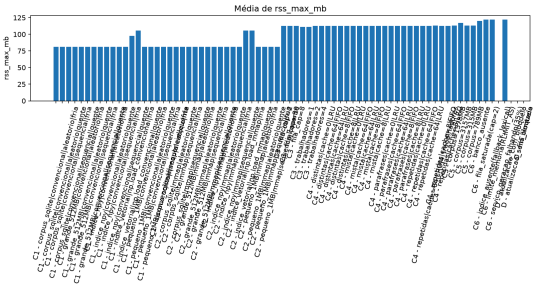


Gráfico de RSS usada o carregamento de índice.

9.2. Experimento C5: Pressão da Memória e Aumento do Corpus

A escalabilidade do sistema sob restrição de memória foi testada variando o tamanho do corpus de 315MB até 3157MB

.

Com o corpus em 315MB, a latência média de consulta foi de 0.02176s e o sistema registrou 22.740 faltas de página menores (minflt).

Ao expandir o corpus para 3157MB, a latência média de consulta aumentou aproximadamente dez vezes, atingindo 0.2118s.

O tempo de alocação também degradou severamente, passando de 0.1308s (315MB) para 1.325s (3157MB), acompanhado de um salto para 120.600 faltas de página menores.

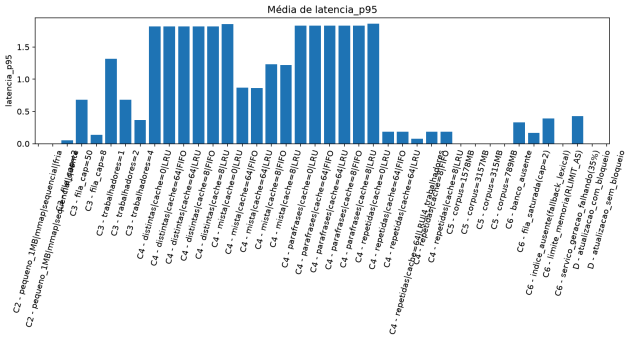


Gráfico de latência quando o corpus sobe de 315MB para 3157MB.

10. Resultados e Análise: Concorrência, Cache e Correção

10.1. Experimento C3: Impacto da Fila e Escalabilidade dos Trabalhadores

A configuração testou como múltiplos produtores e consumidores (trabalhadores) afetam a vazão e a degradação do sistema.

O aumento de 1 para 4 trabalhadores reduziu o tempo total de execução de 5.137s para 1.329s.

A vazão escalou de maneira quase linear, subindo de 38.93 requisições por segundo (1 trabalhador) para 150.5 requisições por segundo (4 trabalhadores).

Quando a capacidade da fila foi restringida a 2, o sistema sofreu contenção severa, resultando em 10.02s de tempo de bloqueio dos produtores.

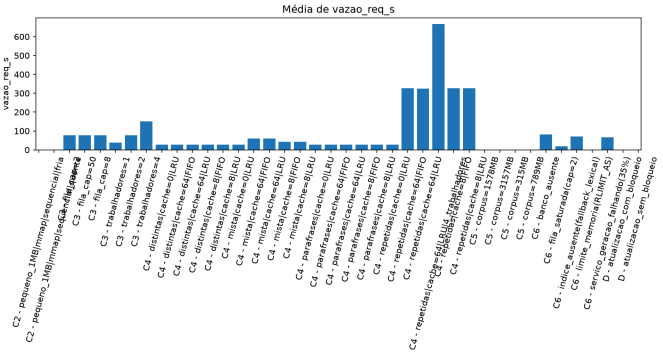


Gráfico de ganho de vazão.

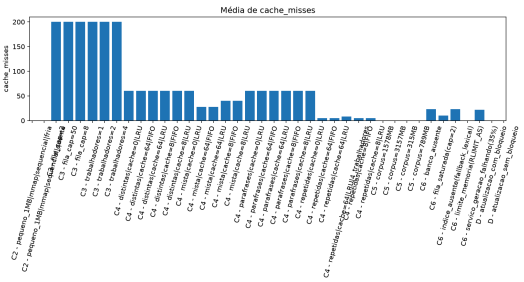
10.2. Experimento C4: Desempenho da Cache Compartilhada

O cache foi implementado para armazenar resultados recentes e avaliar o impacto de consultas repetidas e distintas na latência.

Com consultas repetidas e uma capacidade de cache de 8 ou 64 itens, o sistema obteve 55 acertos (cache_hits) e economizou 0.9167s em relação à execução sem cache.

Para consultas mistas (cache de 64 com LRU), ocorreram 32 acertos e 28 erros, economizando 0.5333s de processamento.

Para consultas com paráfrases ou consultas totalmente distintas, a taxa de acerto foi zero independentemente da política (LRU ou FIFO), indicando que o cache exato não mitiga o custo computacional de entradas variáveis.



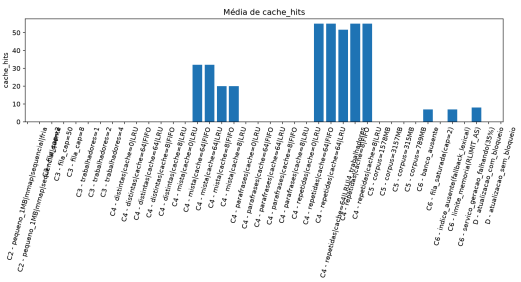


Gráfico de falhas de cache nas cargas distintas e sem cache configurado.

Gráfico de acerto de cache em consultas repetidas e mistas.

10.3. Parte D e C6: Sincronização e Demonstração de Erros Controlados

O experimento demonstrou os riscos de ambientes concorrentes sem mecanismos de exclusão mútua e simulou falhas na ingestão e acesso de dados.

Na configuração atualizacao_sem_bloqueio, ocorreram 30.000 atualizações perdidas devido à condição de corrida.

A implementação de locks de proteção (atualizacao_com_bloqueio) garantiu a consistência dos dados, registrando zero atualizações perdidas, embora tenha introduzido overhead de sincronização.

O teste de fila saturada (capacidade restrita a 2) resultou em 63 timeouts de inserção (put_timeouts) e 30 requisições rejeitadas/perdidas, demonstrando vulnerabilidade à sobrecarga de entrada.

A aplicação de limites rígidos de memória (RLIMIT_AS) provocou a geração de 1 memoryerror, validando a interrupção controlada de recursos.

11. Relação com Sistemas Operacionais e IA Generativa

Aplicações de busca semântica para LLMs dependem diretamente da eficiência da gestão de recursos realizada pelo Sistema Operacional. O SO tem papel principal, atuando no controle de memória virtual, paginação e escalonamento de threads.

Os testes realizados demonstraram que a não utilização de recursos adequados, como o mmap() para eliminação de cópias desnecessárias na RAM e semáforos no gerenciamento de filas, resulta em sobrecarga no subsistema de I/O em disco, elevando o número de page fault e comprometendo a latência das respostas do pipeline RAG.

12. Limitações da Análise

Os resultados obtidos restringem-se às condições do ambiente experimental analisado, apresentando as seguintes limitações:

- **Ambiente Simulad**o: A simulação de tempos de resposta e o uso de modelos locais não reproduzem integralmente a latência de rede e a concorrência na memória gráfica (VRAM) características de clusters de inferência em produção.

- **Escala da Arquitetura**: O emprego de bancos SQLite e matrizes em arquivos locais (.npy) simplifica o gerenciamento em relação a bancos vetoriais distribuídos, os quais adicionam sobrecarga de protocolos de rede e serialização de dados à equação de desempenho.

13. Conclusão

A investigação evidenciou que o desempenho de uma arquitetura de RAG é diretamente influenciado pelas chamadas de sistema (syscalls) e pela estratégia de gerenciamento da concorrência. Mecanismos providos pelo Sistema Operacional, tais como o mapeamento de memória para otimização de leitura e a sincronização por semáforos e mutexes em filas compartilhadas, revelaram-se indispensáveis para assegurar a escalabilidade, a estabilidade e a integridade dos dados sob cargas intensivas de requisições.